



Hadiya Daud Ismail, 64220004
Department: Computer Engineering
University: Ankara Medipol University

Academic Report: Data Federation with PostgreSQL Foreign Data Wrappers

1. Introduction

Modern enterprise systems rarely store all their data in a single monolithic database. Data is distributed across various services, formats, and physical servers. When an application needs to generate a report, analysts typically have to export data from multiple systems and join it externally, or build complex ETL pipelines. This project demonstrates an advanced database capability called **Data Federation**, enabled by PostgreSQL Foreign Data Wrappers (FDW).

2. What are Foreign Data Wrappers?

Foreign Data Wrappers are a PostgreSQL extension of the SQL/MED (Management of External Data) standard. They allow PostgreSQL to connect to external data sources—such as MongoDB, MySQL, another PostgreSQL server, or even flat files like CSVs—and query them as if they were standard relational tables residing locally.

4. Examples of FDW Wrappers

PostgreSQL includes built-in wrappers such as `file_fdw` and `postgres_fdw`, but the wider FDW ecosystem also supports many other external systems. The table below shows examples of wrappers that could be used in real-world data federation scenarios:

FDW wrapper	External source	Typical use case	Notes
file_fdw	CSV or flat files on the database server	Query exported reports or raw files without importing them	Built into PostgreSQL; read-only
postgres_fdw	Another PostgreSQL database	Join data between separate PostgreSQL databases	Built into PostgreSQL; supports reads and some writes
mysql_fdw	MySQL or MariaDB databases	Combine PostgreSQL analytics with data from MySQL applications	Requires an additional extension
oracle_fdw	Oracle databases	Access enterprise or legacy Oracle systems from PostgreSQL	Requires Oracle client libraries
tds_fdw	Microsoft SQL Server and Sybase	Federate data from SQL Server into PostgreSQL	Useful in mixed database environments
mongo_fdw	MongoDB collections	Query document-style data through PostgreSQL	Data type mapping can be more complex than relational wrappers
redis_fdw	Redis key-value data	Read cache or session-style data from PostgreSQL	Best suited for simple key-value access patterns
ogr_fdw	GIS files and spatial formats	Query shapefiles, GeoJSON, spreadsheets, and other OGR-supported sources	Often useful with PostGIS

For this project, file_fdw and postgres_fdw were selected because they are bundled with standard PostgreSQL installations and work well in a Windows development environment without requiring extra binary dependencies.

4. Data Federation Explained

Data Federation is the concept of keeping data in its original, distributed location while providing a unified, virtual view to the user.

- **Local Tables:** Data is stored physically on the disk managed by the current PostgreSQL instance.
- **Foreign Tables:** The table definition exists in PostgreSQL, but the actual data remains in the external system. When a query is executed, PostgreSQL acts as a client, fetches the necessary data from the remote source, and processes it locally.

5. Why I Chose Foreign Data Wrappers

For the technology selection, the primary constraint was the local development environment. The project had to run entirely on a personal Windows laptop with limited available storage. Extensions like TimescaleDB require downloading and manually installing additional binaries, copying DLL files into the PostgreSQL directory, and modifying system configuration files — adding several hundred megabytes of dependencies. Apache AGE is not distributed as a precompiled Windows binary and would have required compiling from source using Visual Studio C++ Build Tools, which is both storage-heavy and error-prone on Windows.

Foreign Data Wrappers, by contrast, are bundled natively with every standard PostgreSQL installation. Both `file_fdw` and `postgres_fdw` required zero additional downloads — they are activated with a single SQL command. This made FDW the most practical choice given the storage and environment constraints, while still being a genuinely advanced PostgreSQL feature not covered in the course syllabus.

Beyond practicality, FDW addresses a real and important database problem: in production systems, data rarely lives in a single database. FDW allows PostgreSQL to act as a federated query engine, joining data across a CSV file and a remote database in a single SQL statement — a capability that is directly relevant to enterprise data architectures.

6. The Project Scenario

In our simulated enterprise scenario, we constructed an architecture utilizing three distinct data sources to represent a fragmented corporate data environment. The project uses synthetic e-commerce data consisting of 500 customers and 5,000 orders to demonstrate the database technology.

- **customers:** A local PostgreSQL table residing in the primary database (`main_company_db`).
- **foreign_orders_csv:** A CSV file (`external_orders.csv`) accessed dynamically via `file_fdw`.
- **orders_remote:** A table located in a completely separate database instance (`sales_remote_db`) accessed via `postgres_fdw`.

Query: `SELECT * FROM public.customers ORDER BY customer_id ASC`

customer_id	customer_name	city	segment	signup_date
1	Prof. Safinaz Korutürk Ülker	Gazelfort	VIP	2025-09-23
2	Akmal Açikel Demir Mansız	Demirfort	Standard	2024-10-24
3	Köker Erdoğan	Port Ekberville	VIP	2025-06-08
4	Gürelcem Yaman	Tanakhaven	Standard	2025-06-04
5	Yurtgüven Okbay Alemdar Şener	West Tagangülport	Basic	2024-11-13
6	Okt. Demiryürek Arsoy	Bilginhaven	Basic	2025-10-18
7	N'zamett'n Zeynelabidin Arsoy	Bağdathaven	Basic	2026-01-04
8	Evrin Hanbiken Yaman Kısakürek	New Tülin	Standard	2024-06-09
9	Taçnur Yıldırım	Güçlüview	Premium	2025-11-17

customers table

Query: `SELECT * FROM public.orders_remote ORDER BY order_id ASC`

order_id	customer_id	order_date	product_category	amount	status
90	1090	2025-09-16	Electronics	2670.28	Processing
91	1091	2026-04-06	Clothing	66.44	Pending
92	1092	2025-09-24	Home	3358.25	Cancelled
93	1093	2026-01-12	Books	1874.84	Shipped
94	1094	2026-03-07	Electronics	4934.59	Cancelled
95	1095	2025-07-23	Sports	3904.58	Cancelled
96	1096	2026-02-01	Books	3950.20	Cancelled
97	1097	2025-12-09	Books	1232.51	Shipped
98	1098	2025-09-02	Sports	461.38	Shipped
99	1099	2025-09-05	Home	4920.67	Processing

orders_remote table

7. Implementation and Execution

The project successfully established federation through the following technical steps:

1. **File Federation (file_fdw):** We enabled the file_fdw extension and created a server definition pointing to the local filesystem. We then mapped a foreign table to the CSV file, allowing PostgreSQL to parse the CSV on-the-fly during execution without importing the rows into the database storage.
2. **Remote Database Federation (postgres_fdw):** We enabled postgres_fdw, defined the remote server (sales_remote_db), and established a user mapping to securely handle credentials. We used the IMPORT FOREIGN SCHEMA command to dynamically link the remote table definition into the local database.

3. **The Join Query:** With the architecture in place, we demonstrated the capability to perform standard SQL JOIN operations connecting the local customers table with both the CSV-backed foreign table and the remote PostgreSQL table. The query planner successfully parsed these unified queries, fetching data from disk and over the network seamlessly based on the shared customer_id.

9. Application Demonstration

To prove the success of the data federation architecture, we built an interactive dashboard using Streamlit. This dashboard allows us to execute federated SQL JOIN queries live and observe the results.

9.1 The Federated Architecture

The screenshot shows a web browser window displaying a Streamlit application. The browser's address bar shows 'localhost:8501'. The application has a dark theme. On the left, there is a sidebar titled 'Architecture' with three items:

- 1. Local Data (customers) Stored normally inside main_company_db .
- 2. CSV External Data (file_fdw) Orders stored in a raw .csv file on the hard drive. Accessed via the foreign_orders_csv pointer.
- 3. Remote PostgreSQL (postgres_fdw) Orders stored in a completely separate database called sales_remote_db . Accessed via the orders_remote pointer.

The main content area has a title 'Data Federation with PostgreSQL Foreign Data Wrappers' and a subtitle 'This demo showcases how PostgreSQL can act as a federated data hub. Instead of migrating all data into a single database, PostgreSQL queries data where it natively lives using Foreign Data Wrappers (FDW).'. Below this, there are two tabs: 'Demo 1: Querying a CSV (file_fdw)' and 'Demo 2: Querying Remote DB (postgres_fdw)'. The 'Demo 1' tab is active, showing the title 'Joining Local Table with External CSV' and the text 'We will join customers (Local Table) with foreign_orders_csv (External CSV file accessed via file_fdw)'. Below this, there is a SQL query in a code block:

```
SELECT
  c.customer_name,
  c.city,
  c.segment,
  o.order_id,
  o.product_category,
  o.amount,
  o.status
FROM customers c
JOIN foreign_orders_csv o
ON c.customer_id = o.customer_id;
```

At the bottom of the code block, there is a button labeled 'Run file_fdw Join Query'.

Description: The application architecture relies on three distinct storage locations. Our local PostgreSQL instance (main_company_db) stores the core customers table. However, it does not store order data. Instead, it maintains two "Foreign Table" pointers: foreign_orders_csv (pointing to a flat text file) and orders_remote (pointing to a completely separate PostgreSQL server).

9.2 Demo 1: Joining Local Data with a Text File (file_fdw)

This demo showcases how PostgreSQL can act as a federated data hub. Instead of migrating all data into a single database, PostgreSQL queries data where it natively lives using Foreign Data Wrappers (FDW).

Demo 1: Querying a CSV file (file_fdw) Demo 2: Querying Remote DB (postgres_fdw)

Architecture

1. Local Data (customers) Stored normally inside main_company_db .
2. CSV External Data (file_fdw) Orders stored in a raw .csv file on the hard drive. Accessed via the foreign_orders_csv pointer.
3. Remote PostgreSQL (postgres_fdw) Orders stored in a completely separate database called sales_remote_db . Accessed via the orders_remote pointer.

Joining Local Table with External CSV

We will join customers (Local Table) with foreign_orders_csv (External CSV file accessed via file_fdw).

```
SELECT
  c.customer_name,
  c.city,
  c.segment,
  o.order_id,
  o.product_category,
  o.amount,
  o.status
FROM customers c
JOIN foreign_orders_csv o
ON c.customer_id = o.customer_id;
```

Run file_fdw Join Query

Querying...

localhost:8501

Architecture

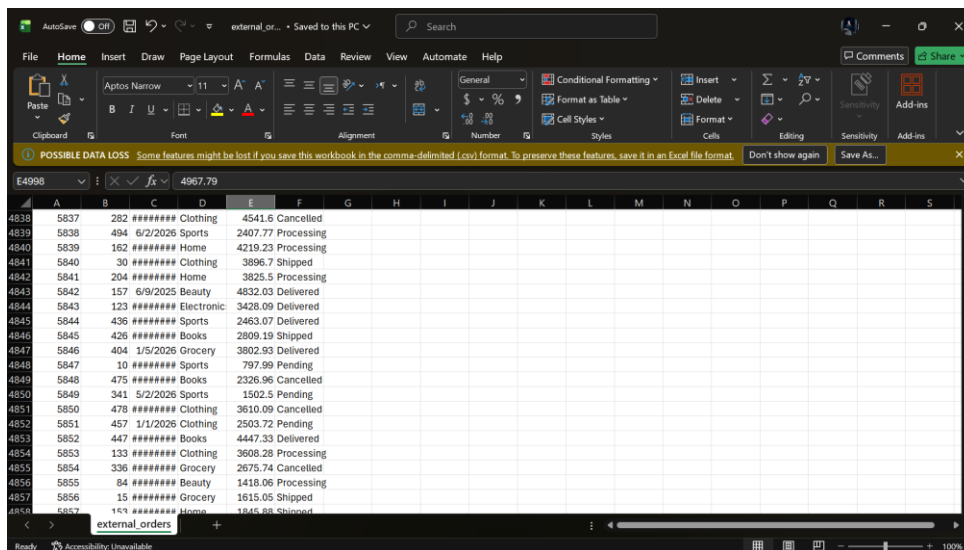
1. Local Data (customers) Stored normally inside main_company_db .
2. CSV External Data (file_fdw) Orders stored in a raw .csv file on the hard drive. Accessed via the foreign_orders_csv pointer.
3. Remote PostgreSQL (postgres_fdw) Orders stored in a completely separate database called sales_remote_db . Accessed via the orders_remote pointer.

```
SELECT
  c.segment,
  o.order_id,
  o.product_category,
  o.amount,
  o.status
FROM customers c
JOIN foreign_orders_csv o
ON c.customer_id = o.customer_id;
```

Run file_fdw Join Query

	customer_name	city	segment	order_id	product_category	amount	status
0	Hindal Yaman	Port Tanguartan	Premium	1001	Clothing	1496.22	Processing
1	Yrd. Doç. Güler Selda Yıldırım Duran	West Cabir	Standard	1002	Books	1523.09	Processing
2	Öğr. Nurgün Semur Ergül Dumanlı	Hezriyeberg	Standard	1003	Sports	0	Pending
3	Doç. Sili Feraholba Arslan Demir	Gülbalıshire	Premium	1004	Electronics	2844.65	Delivered
4	Kerman Gülen	Atlatıad	Standard	1005	Books	2848.85	Shipped
5	Öğr. Karatay Erihan Arsoy	North Yömen	Standard	1006	Sports	9999.9	Pending
6	Sadiye Sirap Gül	Fratıview	Premium	1007	Electronics	119.48	Processing
7	Ökt. Şahdiye Çetin Yılmaz	Rakideside	Premium	1008	Electronics	2104.27	Processing
8	Zeride Caheyne Tanhan	Gülborough	Standard	1009	Clothing	2311.64	Delivered
9	Ökt. Semrin Güzey Akar Sezer	Inonıtown	Basic	1010	Home	1260.12	Cancelled

Description: In this demonstration, we executed a standard SQL JOIN command connecting our local customers table with the foreign_orders_csv pointer. Upon execution, the PostgreSQL engine instantly accessed the raw external_orders.csv text file stored on the Windows hard drive, parsed the text into structured columns in memory, and joined the records. This proves that we can query raw files dynamically without ever running an ETL script to import them.



Description: By altering a value directly inside the raw .csv text file and re-running the query in our dashboard, the updated value was instantly reflected in our SQL results. This confirms the data is being read live from the disk, preventing the "stale data" issue common in traditional data pipelines.

9.3 Demo 2: Joining Local Data with a Remote Database (postgres_fdw)

Architecture

- Local Data (customers) Stored normally inside main_company_db .
- CSV External Data (file_fdw) Orders stored in a raw .csv file on the hard drive. Accessed via the foreign_orders_csv pointer.
- Remote PostgreSQL (postgres_fdw) Orders stored in a completely separate database called sales_remote_db . Accessed via the orders_remote pointer.

Joining Local Table with Remote PostgreSQL

We will join customers (Local Table) with orders_remote (Remote Postgres Table accessed via postgres_fdw).

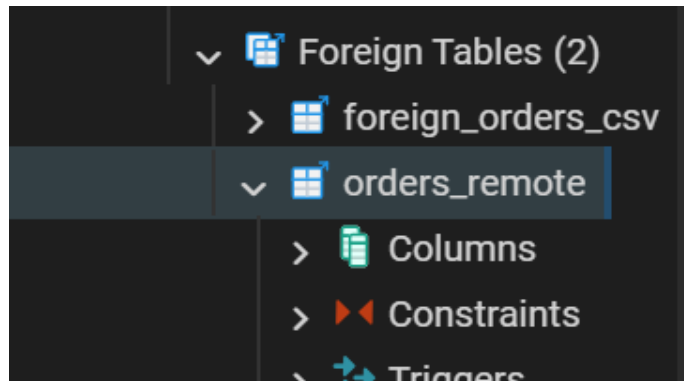
```
SELECT
  c.customer_name,
  c.city,
  c.segment,
  o.order_id,
  o.product_category,
  o.amount,
  o.status
FROM customers c
JOIN orders_remote o
ON c.customer_id = o.customer_id;
```

Run postgres_fdw Join Query

	customer_name	city	segment	order_id	product_category	amount	status
0	Atanur Gül	Lütfiye	Standard	1001	Clothing	3495.61	Delivered
1	Samur İldeğ Alemdar Gül	East Ececiş	VIP	1002	Grocery	1778.57	Cancelled
2	Ertün Demir	Dilcanport	Basic	1003	Home	624.49	Pending
3	Malazgiz Maraz	Çiğdem	Standard	1004	Electronics	4100.36	Pending

Description: In our second demonstration, we executed a JOIN between our local customers table and orders_remote. In the background, our primary database acted as a client, reached out over the network to a completely independent database instance (sales_remote_db), fetched the requested order data, and merged it locally. This illustrates how PostgreSQL can act as a unified hub for a fragmented corporate ecosystem.

9.4 Verification in pgAdmin



FOREIGN TABLES in main_company_db

Description: To verify the physical separation of data, we inspected the database schemas using pgAdmin. As shown above, the `orders_remote` and `foreign_orders_csv` entities are listed strictly under the **Foreign Tables** directory and have a specialized icon. They do not exist as physical tables inside the main database, definitively proving that data federation was successfully achieved.

10. Technology Stack

To construct this federated architecture and the interactive dashboard, the following technology stack was utilized:

- **PostgreSQL 16:** The core open-source Relational Database Management System (RDBMS). In this project, it was utilized not just as a storage mechanism, but as the central federated query engine.
- **Foreign Data Wrappers (SQL/MED):** Native PostgreSQL extensions used to establish the federated links.
 - **file_fdw:** Utilized to parse and query raw CSV text files from the Windows local file system in real-time.
 - **postgres_fdw:** Utilized to establish a network connection to an independent, remote PostgreSQL server to query data dynamically.
- **Python 3 & psycopg2:** The backend programming language and database adapter used to programmatically automate the DDL (Data Definition Language) execution and configure the FDW environments.
- **Pandas & Faker:** Python libraries used to algorithmically generate 5,500 rows of highly realistic synthetic e-commerce data. This ensured the architecture could be tested without relying on static public datasets.
- **Streamlit:** A rapid web application framework used to construct the interactive frontend dashboard. This was crucial for proving that end-user applications can execute federated SQL JOIN queries dynamically as if they were querying a standard local database.

11. Advantages, Limitations, and Security

Advantages

- **No Data Duplication:** Reduces storage costs and eliminates the risk of outdated ETL pipelines.
- **Real-Time Access:** Queries against foreign tables fetch the most up-to-date data directly from the source.
- **Simplicity:** Developers can write standard SQL JOINS instead of complex application-level API routing or integration logic.

Limitations

- **Performance:** Since data must be transferred over the network (or read sequentially from disk for CSVs) during the query execution, it is generally slower than querying heavily indexed local tables.
- **Write Constraints:** While `postgres_fdw` supports INSERT, UPDATE, and DELETE operations back to the remote server, extensions like `file_fdw` act strictly as read-only interfaces.

Security Considerations

When using `postgres_fdw`, user mappings must be securely configured to prevent unauthorized access. The credentials for the remote database are stored inside the primary database's system catalogs and must be restricted using careful role-based access control (RBAC).

12. Conclusion

This project successfully demonstrates that PostgreSQL is not just a relational database, but a powerful data federation hub. By utilizing Foreign Data Wrappers, organizations can abstract away the complexity of distributed systems and treat disparate data sources as a single, unified relational schema.

